Capstone Milestone 03

Programmatic Core Migration Report

Arad Fadaei, Mahboobeh Yasini, Johnpaul Tamburro

Version 3.0, March 12, 2026

Table of Contents

Chapter 1. Milestone Goal

The Goal: Transition core gameplay systems away from heavy LLM dependence and toward deterministic, programmatic logic while preserving optional LLM capabilities for hybrid and narrative-enhanced modes.

Why this milestone matters: Previous iterations validated the LLM-driven gameplay concept, but mechanics like intent classification and room expansion were still tightly coupled to model responses. This created latency, consistency, and reliability concerns for foundational gameplay.

Chapter 2. Summary of Completed Work

2.1. 1) Runtime Mode Configuration (Programmatic First)

We introduced runtime flags that control how the engine behaves:

- `GAME_MODE=programmatic|hybrid|llm`
- `ENABLE_LLM_NARRATION=true|false`

This enables deterministic operation by default while keeping migration-friendly paths for mixed mode and strict LLM mode.

2.2. 2) Deterministic Intent Classification

A dedicated parser now handles common game actions without an LLM round-trip:

- move (including direction aliases like `n`, `north`, `left`, `up`)
- look/examine
- take/loot
- attack/fight
- inventory

The parser includes:

- normalization and tokenization
- direction extraction
- target extraction with stopword filtering
- explicit unknown fallback for unsupported inputs

2.3. 3) Programmatic Turn Resolution with Optional LLM Fallback

Turn processing now branches by runtime mode:

- **programmatic:** deterministic parser only
- **hybrid:** parser first, LLM fallback only when parser returns unknown
- **llm:** LLM-only intent classification

Narrative output behavior is now:

- canonical gameplay response always produced from deterministic logic

- optional LLM narrative embellishment only when enabled

2.4. 4) Procedural Generation Refactor (Biome + Difficulty)

Programmatic generation was expanded from simple random content to structured tables:

- biome inference from theme (`mine`, `crypt`, `arcane`, `volcanic`)
- biome-specific descriptor/feature/item/enemy content tables
- room difficulty scaling by room id progression
- difficulty-based item/enemy spawn rolls
- enemy HP scaling based on difficulty band

Result: generation quality is more coherent and progression-aware without requiring LLM calls.

2.5. 5) API Contract Alignment for Frontend Diagnostics

Frontend diagnostics expected `/api/generate` and `/api/classify`, but those endpoints were previously missing.

Implemented:

- `POST /api/generate`
  - uses LLM if available
  - returns deterministic fallback text in pure programmatic mode
- `POST /api/classify`
  - deterministic classification baseline
  - hybrid/llm integration path when provider is available

Health metadata now includes all active endpoint references used by the frontend diagnostics screen.

2.6. 6) Test Suite Introduction

A new backend test suite was added under `src/api/tests`:

- intent parser tests
- game turn behavior tests (move/take/attack)
- programmatic generator tests

Test dependencies added:

- pytest
- pytest-asyncio

Chapter 3. Validation Results

3.1. Automated Tests

Executed from `src/api`:

- `python -m pytest -q`
- **Result:** `11 passed`

3.2. Observed Warnings (Non-blocking)

- Pydantic v2 warning about class-based `Config` deprecation.
- LangChain warning regarding pydantic v1 compatibility on Python 3.14+.

These warnings did not block tests and are deferred to a compatibility/cleanup pass.

Chapter 4. Architecture Impact

4.1. Reduced LLM Critical Path

Core gameplay can now run entirely in deterministic mode:

- intent classification no longer requires a model
- room/theme generation can complete without model availability
- API startup no longer hard-fails when LLM is unavailable (except strict llm mode)

4.2. Improved Reliability and Cost Control

- fewer model calls for routine turns
- lower token usage for world generation and command parsing
- faster and more predictable turn processing

Chapter 5. Risks and Known Gaps

5.1. 1) Combat Depth Still Minimal

Combat currently removes matched enemies immediately upon successful attack intent. A full stat-driven combat model remains future work.

5.2. 2) Procedural Content Needs Balancing Pass

Biome tables and difficulty rolls are functional, but content rarity and encounter pacing need playtest tuning.

5.3. 3) LLM Compatibility on Python 3.14

Current warning indicates ecosystem lag for some dependencies. Not a blocker for deterministic mode, but relevant for long-term hybrid/llm stability.

Chapter 6. Next Milestone Focus (Proposed)

6.1. Priority 1: Progression and Balance

- rarity tiers per biome
- weighted loot progression
- encounter composition rules (single elite vs. multiple weak enemies)

6.2. Priority 2: State-rich API Responses

- include explicit room/player snapshot in game turn responses
- support richer frontend rendering without extra round-trips

6.3. Priority 3: CI and Quality Gates

- add CI workflow to run backend tests on PRs
- add lint/type checks for backend and frontend

6.4. Priority 4: Technical Debt Cleanup

- migrate pydantic settings config style to v2 pattern
- evaluate langchain dependency compatibility matrix for Python 3.14

Chapter 7. Conclusion

Milestone 03 successfully establishes a reliable programmatic gameplay core while preserving optional LLM augmentation paths. The project now has a stronger foundation for performance, determinism, and testability, enabling faster iteration on game design and user-facing features in subsequent milestones.